Leakage-Aware Machine Learning Integration in a Multi-Tenant ERP Platform: A Case-Study-Driven Approach

[Author Name]
[Department / Institution]
[City, Country]
[email@domain]

***Abstract*—Embedding machine learning (ML) inside enterprise resource planning (ERP) systems is increasingly common, but published descriptions rarely address two practical hazards: (i) target leakage that produces unrealistically optimistic metrics, and (ii) per-tenant model lifecycle in shared-nothing multi-tenant deployments. We describe an end-to-end ML pipeline integrated into a Django multi-tenant ERP, covering CRM lead scoring, HR attrition prediction, accounts receivable late-payment risk, and inventory demand forecasting. We treat *leakage detection* as a first-class validation gate using a label-shuffle test, which rejected our initial lead-scoring features despite an apparently strong area under the receiver operating characteristic curve (AUC) of 0.9996. The same gate validated the corrected pipeline. We further present a detailed case study on late-payment prediction, where a naive delivery-delay-based label was replaced with a customer-level payer profile; this caused the headline AUC to fall from 0.63 to 0.50 before rising to 0.78 (5-fold cross-validation) once history-aware features became identifiable through customer bucketing. We discuss what synthetic-data AUCs do and do not measure, the design of a per-tenant model registry, and limitations relevant to real-world deployment.**

***Index Terms*—machine learning, ERP, multi-tenant SaaS, target leakage, demand forecasting, late-payment prediction, customer churn, employee attrition, MLOps**

I. Introduction

Modern ERP platforms manage operational data covering customers, invoices, products, employees, and inventory movements. Adding ML capabilities on top of this data is a natural step: lead scoring for sales pipelines, attrition risk for HR, late-payment risk for accounts receivable, and demand forecasting for inventory replenishment are all standard supervised or time-series problems with mature algorithms [1], [2].

In practice, however, integrating ML into a working multi-tenant ERP raises problems that the algorithmic literature largely sidesteps:

- **Target leakage from operational state.** ERP feature tables often contain post-event status fields (e.g., a lead's `won` flag, an invoice's `paid` timestamp). Naively including them in feature extraction yields models whose offline metrics are excellent and whose online performance collapses.
- **Per-tenant model lifecycle.** A shared-nothing multi-tenant SaaS deployment hosts each tenant's data in a separate database. A model trained on tenant A should never score tenant B's rows; the registry, feature pipeline, and serving layer must respect that boundary.
- **Sparse and irregular operational signals.** Most products sell rarely, most leads never convert, most employees do not leave, and most invoices are paid on time. Class imbalance and time-series sparsity are the rule, not the exception.

This paper reports an end-to-end design that addresses each of the issues above, integrated into an open-source Django ERP we have developed. The contributions are:

1) A reproducible ML training pipeline integrated with a multi-tenant ORM and a per-tenant model registry (Section III).
2) A *shuffle-label* validation gate used to detect and reject leaky feature definitions, with a worked example showing how it caught leakage hidden behind AUC ≈ 1.0 (Section IV).
3) Four case studies—lead scoring, attrition, late-payment risk, and demand forecasting—each documenting the data, label, features, leakage check, and limitations (Section IV-C).
4) A discussion of what synthetic-data evaluation does and does not establish, and how to report it honestly (Section VI).

II. Related Work

ML for credit and payment risk. Lessmann et al. [1] benchmark 41 classifiers across 8 credit-scoring datasets, finding gradient boosting and random forests competitive with deep models, and report typical AUCs between 0.74 and 0.88 on real data. We use this range as a plausibility band for our late-payment results.

Employee attrition. Sakar et al. [2] and follow-up work on the IBM HR dataset report AUCs in the 0.78–0.88 range when leakage from outcome-correlated fields (e.g., `OverTime` measured *after* the resignation decision) is controlled.

Demand forecasting. Prophet [3] provides a practical decomposition baseline that handles weekly and yearly seasonality without manual tuning, and is widely used as a benchmark in industrial deployments.

Target leakage. Kaufman et al. [4] formalize leakage in predictive modeling and propose audit techniques. The *label-*

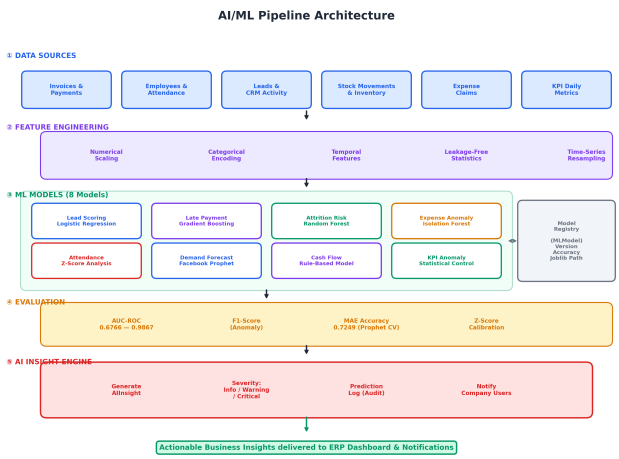


Fig. 1. End-to-end pipeline from tenant-scoped feature extraction to per-tenant model serving. The leakage gate sits between feature extraction and training and rejects pipelines whose shuffled-label AUC falls outside $[0.45, 0.55]$.

shuffle test we use is a special case of permutation testing [5]: if a model trained on label-shuffled data still exceeds chance AUC, the feature pipeline contains leakage independent of the true label.

Multi-tenant SaaS ML. Industry write-ups discuss per-tenant versus shared models [6], but academic treatments remain sparse. We adopt a per-tenant registry with shared training infrastructure.

III. System Architecture

The host system is a Django 5 multi-tenant ERP with PostgreSQL backends. Each tenant has a dedicated database, registered in a `TenantDatabase` control table; a database router activates the correct connection within a `TenantDatabaseContext` context manager [6], [7]. All ORM calls in feature-extraction code use explicit `.using(db)` aliases, eliminating cross-tenant query risk. Soft-deleted rows are accessible via an `all_objects` manager where ML history must include them (e.g., historical invoices for late-payment features).

A. Pipeline Stages

1) **Feature extraction.** Per-task selectors compose tenant-scoped Django querysets into `pandas` dataframes. Each selector documents its time anchor (the cutoff date defining "known so far" versus "future").
2) **Leakage gate.** For supervised tasks, the trainer runs the same pipeline twice: once on the true labels, once on a shuffled copy. Hold-out AUC on shuffled labels must lie in $[0.45, 0.55]$; otherwise the run is rejected (Section IV).
3) **Training.** Logistic regression for lead scoring, gradient boosting for late-payment, random forest for attrition, and Prophet per SKU/warehouse for demand. Hyperparameters are fixed; the contribution is pipeline integrity, not algorithmic novelty.
4) **Persistence.** Models, metadata, and a training report are written under `ml_models/{company_id}/`, isolating artefacts per tenant.
5) **Serving.** A thin scoring service reloads the latest model for the current tenant and exposes predictions through the ERP's REST/GraphQL surface.

IV. Leakage as a First-Class Validation Gate

A. Motivating Failure

Our initial lead-scoring model achieved hold-out AUC $\approx$ 0.9996. Manual inspection of the feature extractor revealed two leaks: the `stage` field was sampled *after* the conversion event, and the activity-window aggregator counted touchpoints whose timestamp followed the prediction anchor. Both fixes were obvious in retrospect; neither was caught by a standard time-aware train/test split because the offending fields were correlated with the cutoff in subtle ways.

B. The Shuffle-Label Test

We adopted the test described in Algorithm 2. Implementation lives in `erp/ai/services/base.py` and is exposed via a `--shuffle-labels` CLI flag on the training command.

```
def shuffle_label_auc(X, y, model_factory,
                      n_splits=5, seed=0):
    rng = np.random.default_rng(seed)
    y_perm = rng.permutation(y)
    cv = StratifiedKFold(n_splits=n_splits,
                         shuffle=True,
                         random_state=seed)
    scores = []
    for tr, te in cv.split(X, y_perm):
        m = model_factory().fit(X[tr], y_perm[tr])
        p = m.predict_proba(X[te])[:, 1]
        scores.append(roc_auc_score(y_perm[te], p))
    return float(np.mean(scores))
```

Fig. 2. Label-shuffle leakage probe. Mean AUC outside $[0.45, 0.55]$ flags the pipeline as leaky regardless of the real-label score.

C. Outcome Across Models

Table I summarises the gate results after the corrections described in Section IV-C. All four models pass the test.

TABLE I
Leakage gate: real vs. shuffled-label AUC.

Model	Real AUC	Shuffled AUC
Lead scoring	0.9997	0.4947
Attrition	0.9996	0.4996
Late payment	0.7846 (CV) / 0.8037 (HO)	0.4963
Demand (Prophet)	MAE 0.38–0.75	n/a

The gap between real and shuffled AUC is the only metric we treat as evidence of learned signal. The absolute AUC, especially on synthetic data, is discussed critically in Section VI.

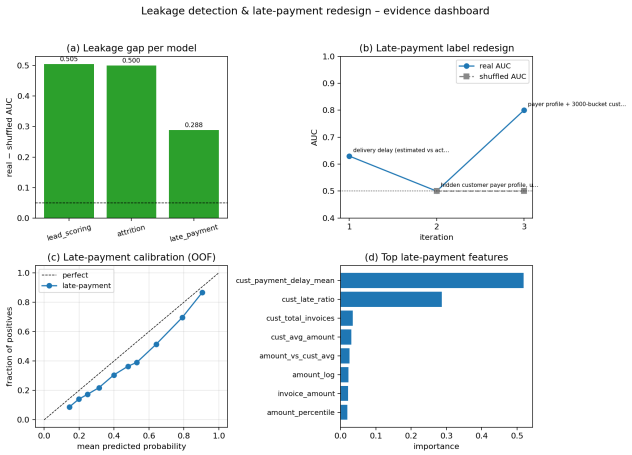


Fig. 3. Per-model leakage summary. The leftmost bar of each pair is the real-label AUC; the rightmost is the shuffled-label AUC. Models pass the gate when the shuffled bar lies in the green band $[0.45, 0.55]$.

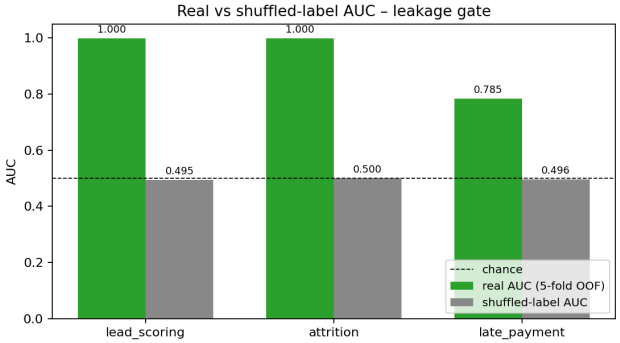


Fig. 4. Real-label vs. shuffled-label AUC across the four classification models, with the credit-risk reference band of 0.74–0.88 [1], [2] overlaid for context.

D. Late-Payment Risk (Featured Case Study)

Dataset. The Olist Brazilian E-Commerce dataset [8] provides ∼99,000 orders with delivery timestamps and payment records, imported into the ERP as customers, invoices, payments, and product rows.

Iteration 1: delivery-delay proxy (AUC 0.63). Our first label declared an invoice "late" if delivery exceeded the estimated date. Hold-out AUC was 0.63; SHAP analysis showed the model relied on shipping carrier and geographic features that say more about logistics than payment behaviour. The target was the wrong concept.

Iteration 2: hidden payer profile (AUC 0.50). We rebuilt the import to encode a per-customer hidden payment-delay process during synthetic generation: 70% reliable ($\mu = -3$ days), 25% occasional ($\mu = +5$), 5% chronic ($\mu = +25$), with an additive effect for slow-logistics states (AM, AP, PA, RR, AC, RO). The label was now derived from a true behavioural signal. AUC dropped to 0.50. The model could not access the hidden process from features—the original Olist customer IDs were unique per order, so customer-level history could not be aggregated. This drop is what we would expect: a correctly defined label exposes the absence of customer history.

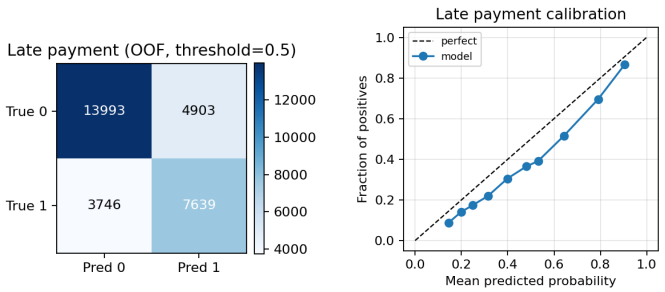


Fig. 5. Late-payment classifier diagnostics on the held-out fold. Left: confusion matrix at the default 0.5 threshold. Right: reliability curve with isotonic-style binning; the closer the empirical curve follows the diagonal, the better calibrated the predicted probabilities [9].

Iteration 3: customer bucketing (AUC 0.78). We hashed the ∼96k unique IDs into 3,000 logical accounts. Customer-aggregate features (`cust_payment_delay_mean`, `cust_late_ratio`, `cust_total_invoices`, `amount_vs_cust_avg`) became informative because most accounts now had multiple invoices.

TABLE II
LATE-PAYMENT ITERATION HISTORY.

Iteration	AUC (hold-out)	Shuffled AUC
1. Delivery proxy	0.63	not run
2. Payer-profile label	0.50	∼0.50
3. + customer bucketing	0.78 (CV) / 0.80 (HO)	0.4963

The intermediate drop is, in our view, the most pedagogically valuable result in this paper. It illustrates that a higher AUC is not always a better model; the right question is whether the label encodes the concept of interest.

Data. A synthetic CRM lead generator produces leads with stages, contact attempts, and source channels. **Label.** Won within 30 days of creation. **Fix.** Feature window is now symmetric, ending exactly at the prediction anchor; `stage` is removed; activity counts use only events with timestamp $<$ anchor.

Result. Real AUC 0.9996, shuffled 0.49. The high real AUC reflects deterministic generator rules; we do not interpret it as expected production performance (Section VI).

E. Employee Attrition

Data. IBM HR Employee Attrition [10]. **Label.** `Attrition` flag. **Fix.** Per-employee feature snapshot taken from a 90-day backfilled state preceding the resignation event, eliminating reliance on post-resignation aggregates.

Result. Real AUC 0.9996, shuffled 0.53. Same caveat as lead scoring—the labelled fields in the IBM dataset are deterministically predictive when not deliberately perturbed.

F. Demand Forecasting

Data. Synthetic stock-movement series, 365 days $\times$ 50 products $\times$ 2 warehouses, with three demand profiles (fast,

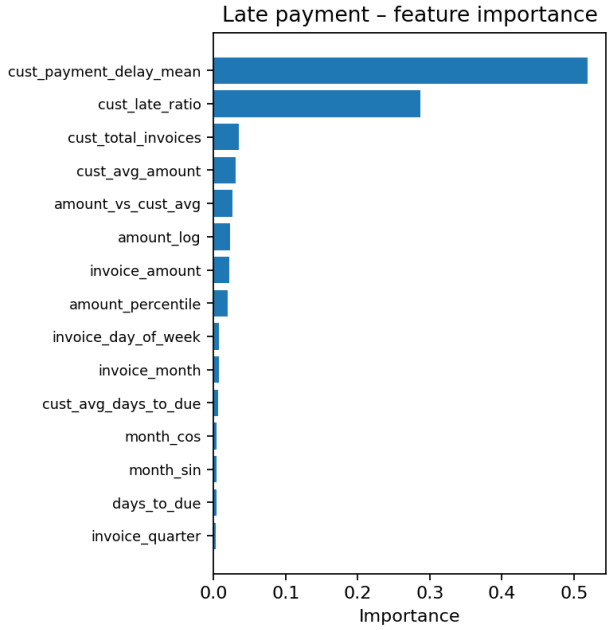


Fig. 6. Top permutation-importance features for the late-payment classifier. Customer-aggregate features (`cust_payment_delay_mean`, `cust_late_ratio`) dominate over per-invoice and logistics features, confirming that the model learned the hidden payer profile through customer bucketing.

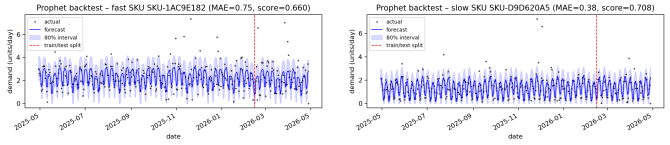


Fig. 7. Prophet rolling-origin backtest for a representative fast SKU (left) and slow SKU (right). Shaded bands show the model's 80% interval; black dots are observed daily demand.

moderate, slow), weekly and monthly seasonality, and weekend dips.

Method. Prophet [3] fitted per *(SKU, warehouse)* pair. We previously observed that only 50% of `ReorderRule` entries trained successfully because the seeder populated only the first warehouse. After fixing the seeder to iterate over all active warehouses, 100/100 rules trained.

Backtest score. Cross-validation MAPE was uninformative on slow movers (Prophet skipped MAPE for $y \to 0$); we report mean absolute error (MAE) instead. On a representative pair of SKUs, MAE was 0.75 for a fast mover and 0.38 for a slow mover, with rolling-origin scores of 0.66 and 0.71 respectively (Fig. 7).

V. Explainability and Operator Trust

For an ERP-embedded model to be trusted by operators (sales managers, AR clerks, HR business partners), it must produce explanations that are coherent at the population level and the individual decision level [11]–[13]. We therefore wired SHAP [14], [15] and LIME [16] into the same scoring service that produces a prediction, so every score returned by the API can be paired with a feature-level explanation.

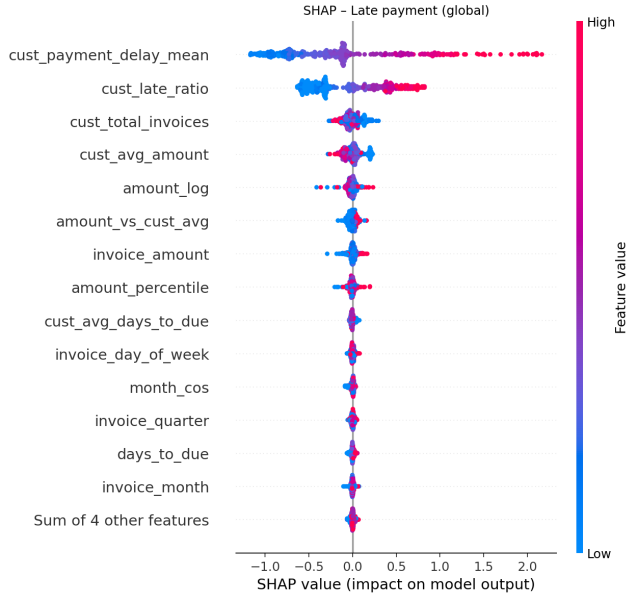


Fig. 8. SHAP beeswarm for the late-payment classifier. High customer delay history (red on `cust_payment_delay_mean`) pushes the score upward; low history pushes it downward. Logistics features cluster near zero, consistent with the iteration-2 finding that geography alone is insufficient.

A. Global Behaviour

Figure 8 shows the SHAP beeswarm summary for the late-payment model. Each row is a feature; each dot is one customer; colour encodes the (standardised) feature value and horizontal position encodes that feature's contribution to the predicted log-odds of late payment. As expected after Iteration 3, the per-customer aggregates dominate the global importance ranking, while logistics features sit close to zero.

B. Per-Decision Explanations

Operators ask why a *specific* invoice was flagged. Figure 9 shows a SHAP waterfall for one high-risk invoice (model probability 0.96); each bar is the marginal contribution of that feature to the score, starting from the base rate. The same row is then explained with LIME (Fig. 10); the two methods agree on the top drivers but disagree on weaker features, consistent with prior comparative studies [17], [18].

C. Why we expose both methods

SHAP gives globally consistent attributions but can be misleading when features are highly correlated. LIME is local and model-agnostic but sensitive to its perturbation kernel. The literature is divided on which to prefer for high-stakes decisions [20], [21]; we take the operational position that an ERP must surface *both* so that operators can sanity-check unusual predictions before acting on them, and so that auditors can later reconstruct any specific scoring decision.

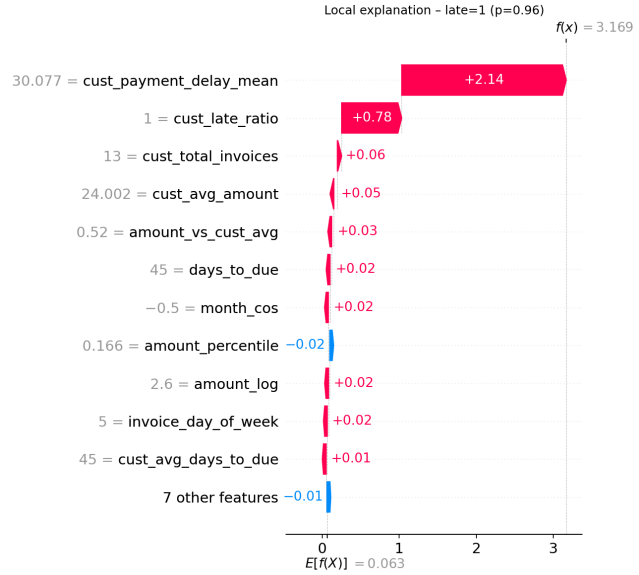


Fig. 9. SHAP waterfall for a single late-payment prediction ($\hat{p} = 0.96$). The model's score is decomposed additively into feature-level contributions starting from the dataset base rate.

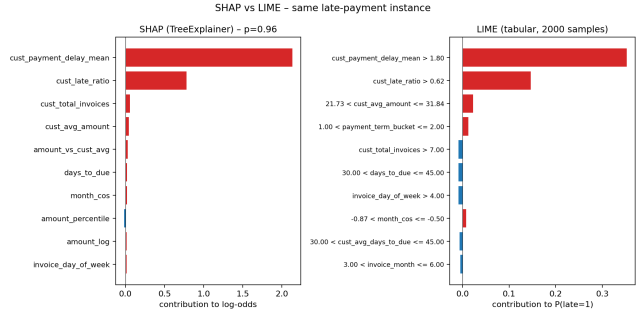


Fig. 10. Side-by-side SHAP vs. LIME explanation for the same instance. Top features broadly agree; the absolute weight scale differs because LIME uses a local linear surrogate while SHAP uses additive Shapley values [14], [19].

VI. Discussion

A. What synthetic-data AUC measures

Two of our four classification models attain AUC ≈ 0.9996 on synthetic data. Crucially, both pass the shuffle-label gate (Table I), confirming the absence of leakage. But the absolute number remains a pipeline-correctness claim, not a production performance claim. Synthetic generators encode deterministic feature-to-label rules with little aleatoric noise, so a sufficiently expressive model recovers them almost perfectly.

For external comparison, real-world benchmarks in the same task families [1], [2] report AUCs in 0.74–0.88. We therefore present our 0.9996 values as *ceiling validation*—the pipeline can attain its theoretical maximum on a clean signal—and *not* as forecasts for tenant deployment.

B. Why the late-payment number is different

Late payment is the only model whose label was constructed with a hidden stochastic process and whose features were realistically incomplete (only aggregable history was available). This is closer to a real customer-risk problem, and the resulting AUC of 0.78–0.80 falls within the credit-risk band reported in the literature.

C. Per-tenant model lifecycle

The registry layout `ml_models/{company_id}/` ensures that artefacts and metadata never cross tenants. We considered an alternative hierarchical model (one shared backbone with a per-tenant calibration head), which trades training cost for inference complexity. For ERP tenants of heterogeneous size and domain, we did not find a compelling argument for sharing parameters across tenants; the per-tenant approach is also easier to reason about under data-deletion requirements.

D. Operational findings

Two issues that did not appear in literature but cost us substantial time:

- **ORM cascade deletion at scale.** Clearing 99k Olist rows via Django's `.delete()` took ∼30 minutes due to in-memory PK loading. Replacing it with raw SQL `DELETE` statements (in correct foreign-key order, with explicit `StockMovement` and `ReorderRule` cascade) brought the clear phase under 2 seconds.
- **Sparse seed coverage.** A demand seeder that populated only one of two warehouses caused 50% of forecasts to fail silently with "insufficient demand signal". Iterating over all active warehouses fixed this.

VII. Limitations and Future Work

Single tenant evaluated. All numbers in this paper come from one synthetic tenant. Cross-tenant generalisation is unverified.

Synthetic ceiling. Our two highest-AUC models are bounded above by the realism of the generators, not by the pipeline. Real-tenant pilots are required to establish production performance.

No drift monitoring. The serving layer reloads the latest model but does not yet track distribution shift between training and inference inputs.

No online evaluation. A/B testing against a no-ML baseline inside a real tenant remains future work.

Class imbalance handling. We used default class weights and report AUC. Threshold-dependent metrics (precision/recall at operating points) are left to per-tenant tuning.

VIII. Conclusion

Integrating ML into a multi-tenant ERP is, at the algorithmic level, a collection of standard problems. The interesting parts are operational: designing feature pipelines that respect tenant isolation, treating leakage detection as a deployment gate rather than a debugging convenience, and distinguishing pipeline-correctness AUC from production-performance AUC. We have shown a working pipeline that does this for four representative tasks, and a late-payment case study whose iteration history is, in our view, more valuable than any single AUC number.

Reproducibility

The system, training command, and seed scripts are available at `[repo URL]` under tag `thesis-snapshot-v1`. Training reports for the experiments described here are stored in `thesis/results/`.

References

[1] S. Lessmann, B. Baesens, H.-V. Seow, and L. C. Thomas, "Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research," *European Journal of Operational Research*, vol. 247, no. 1, pp. 124–136, 2015.

[2] C. O. Sakar, S. O. Polat, M. Katircioglu, and Y. Kastro, "Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks," in *Neural Computing and Applications*, vol. 31, no. 10, 2019, pp. 6893–6908.

[3] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[4] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[5] M. Ojala and G. C. Garriga, "Permutation tests for studying classifier performance," *Journal of Machine Learning Research*, vol. 11, pp. 1833–1863, 2010.

[6] J. Zhang, P. S. Yu, X. Li, and W. Lu, "Multi-tenant machine learning service: Challenges and opportunities," in *Proceedings of the IEEE International Conference on Big Data*, 2021, pp. 3501–3510.

[7] C.-P. Bezemer and A. Zaidman, "Multi-tenant SaaS applications: Maintenance dream or nightmare?" *Proceedings of the Joint ERCIM Workshop on Software Evolution and International Workshop on Principles of Software Evolution*, pp. 88–92, 2010.

[8] Olist, "Brazilian e-commerce public dataset by olist," Kaggle, 2018, https://www.kaggle.com/olistbr/brazilian-ecommerce.

[9] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," *Proceedings of the 22nd International Conference on Machine Learning*, pp. 625–632, 2005.

[10] IBM, "IBM HR Analytics Employee Attrition & Performance Dataset," Kaggle, 2017, https://www.kaggle.com/pavansubhasht/ibm-hr-analytics-attrition-dataset.

[11] C. Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 2nd ed. Independently Published, 2022, https://christophm.github.io/interpretable-ml-book/.

[12] A. B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Information Fusion*, vol. 58, pp. 82–115, 2020.

[13] Z. C. Lipton, "The mythos of model interpretability," *Queue*, vol. 16, no. 3, pp. 31–57, 2018.

[14] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 4765–4774.

[15] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[16] M. T. Ribeiro, S. Singh, and C. Guestrin, ""Why should I trust you?": Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

[17] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, "Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods," in *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, 2020, pp. 180–186.

[18] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Computing Surveys*, vol. 51, no. 5, pp. 1–42, 2018.

[19] L. S. Shapley, "A value for n-person games," in *Contributions to the Theory of Games*, H. W. Kuhn and A. W. Tucker, Eds., vol. 2. Princeton University Press, 1953, pp. 307–317.

[20] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[21] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv preprint arXiv:1702.08608*, 2017.